The Default LoRA Rank Is Not the Blind Spot: Rank Needs Track the Learning Rate, Not the Task Type

NeuriCo

Abstract

Most LoRA recipes hard-code a rank of 8 or 16. A common belief holds that this default suits easy tasks such as sentiment classification but under-serves generative tasks such as math and code, which are thought to need rank 32 or more. We test this belief with a controlled sweep of about 200 adapters: six tasks (four classification tasks, GSM8K, and Python code) by eight ranks (1–128) on QWEN2.5-0.5B (two seeds) and QWEN2.5-1.5B, with a fixed data budget of 4,000 examples, plus a learning-rate sweep and a cheap predictor based on the spectrum of the full-weight gradient at initialization. At a fixed learning rate of 2×10^{-4}, the best rank does vary from 1 to 128, but in the opposite direction from the belief. GSM8K and code do best at low rank and degrade at rank 128 (GSM8K: 37.3% at r=1 vs. 30.1% at r=128). The only task where rank 8 loses significantly after Holm correction is 6-way emotion classification (+2.6 pp at 0.5B, +2.2 pp at 1.5B), and this gap disappears once the learning rate is tuned per rank: rank 8 reaches 92.8%, matching ranks 32 and 128. The gradient-spectrum statistic orders tasks consistently across model sizes, but it is *negatively* correlated with the empirical knee rank (Spearman $\rho = -0.74$, $p = 0.014$, $n = 12$). In the small-data supervised fine-tuning regime we study, rank 8 is not a blind spot. Pairing a large rank with an untuned learning rate is the real risk.

1 Introduction

Low-rank adaptation (LoRA) [Hu et al., 2022] is the default way to fine-tune large language models on a budget. Nearly every recipe fixes the rank at r=8 or r=16, usually with $\alpha = 2r$, and rarely justifies the choice. **Is this default a blind spot?** If the rank a task needs varied by an order of magnitude, practitioners would be leaving accuracy on the table for "hard" tasks and wasting memory on easy ones.

There are good reasons to suspect such a blind spot. The original LoRA work found that very low ranks suffice for GPT-3 adaptation [Hu et al., 2022], and fine-tuning has a low intrinsic dimension [Aghajanyan et al., 2021]. On the other hand, later work reports that code and math need much higher rank in continued pre-training [Biderman et al., 2024], and adaptive methods allocate rank unevenly across layers [Zhang et al., 2023]. The resulting folk wisdom is simple: sentiment needs rank ≈ 4, while code needs rank ≥ 32.

However, two problems make this folk wisdom hard to check. First, rank comparisons are rarely controlled: data budget, optimizer, and evaluation change between studies. Second, under the standard $\alpha \propto r$ scaling, rank and effective step size are coupled [Kalajdzievski, 2023]. A comparison across ranks at one learning rate therefore mixes "how much capacity a task needs" with "how large a step the learning rate gives at that rank". Finally, even if rank needs did differ, there is no cheap way to predict them before training.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

We run a controlled study to separate these factors. We hold the data budget (4,000 examples, one epoch), the optimizer, and the LoRA parameterization fixed. We then sweep eight ranks (1 to 128) on six tasks that range from binary sentiment to math word problems and Python code, on two model sizes of one family (QWEN2.5-0.5B with two seeds, QWEN2.5-1.5B with one). We add a learning-rate sweep on three tasks to remove the rank–learning-rate confound. We also test a training-free predictor: the effective rank of the full-weight gradient at the pretrained weights. We pre-register four hypotheses: rank needs vary $\geq 8\times$ across tasks (H1), r=8 costs 3–8 pp on high-rank tasks (H2), the knee rank shrinks with model size (H3), and the gradient statistic predicts the knee with Spearman $\rho > 0.7$ (H4).

Our results mostly refute these hypotheses. At a fixed learning rate, the best rank does span 1 to 128, but the ordering is reversed: GSM8K and code are the *lowest*-rank tasks, and both degrade at r=128 (GSM8K falls from 37.3% at r=1 to 30.1%; code falls from 28.8% to 22.7%, below the 27.0% zero-shot baseline). The only significant r=8 gap after Holm correction is on 6-way EMOTION classification (+2.6 pp at 0.5B, +2.2 pp at 1.5B), and it vanishes when the learning rate is tuned per rank: at learning rate 5×10^{-4}, r=8 reaches 92.8% and beats r=32 and r=128. On EMOTION, the learning rate moves r=8 accuracy by 15 pp, while rank moves it by at most 3 pp at the best learning rate. The gradient statistic separates tasks cleanly (code $\gg$ GSM8K $>$ BOOLQ $>$ AG NEWS $>$ EMOTION $>$ SST-2), yet it *anti*-predicts the knee ($\rho = -0.74$, $p = 0.014$).

Our contributions are:

- We conduct a controlled sweep of about 200 LoRA adapters across 6 tasks, 8 ranks, and 2 model sizes. We show that at a fixed learning rate, rank needs run opposite to the common intuition: math and code prefer rank 1–16, and large ranks cause collapses of 7–11 pp.
- We quantify the cost of the r=8 default with paired bootstrap tests and cross-fitting across seeds. The cost is at most 2.6 pp, and it is significant only on EMOTION.
- We show with a learning-rate sweep that the apparent rank effect is mostly a learning-rate effect under $\alpha = 2r$. Once the learning rate is tuned, r=8 is the best rank or within 0.8 pp of it on all three tasks in the sweep.
- We evaluate a gradient-spectrum predictor of rank needs and find a significant *negative* correlation with the empirical knee. It lands within a factor of 2 of the knee for 0 of 12 task–model pairs, compared with 6 of 12 for always predicting r=8.

Section 2 reviews related work, section 3 describes our setup, section 4 presents results, section 5 interprets them and lists limitations, and section 6 concludes.

2 Related Work

Low-rank adaptation and rank choice. LoRA [Hu et al., 2022] freezes the pretrained weights and learns a low-rank update $\Delta \boldsymbol{W} = \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}$. Hu et al. [2022] report that ranks as small as 1–8 match full fine-tuning on GPT-3 for several NLU and NLG benchmarks. AdaLoRA [Zhang et al., 2023] allocates the rank budget across weight matrices using importance scores, which suggests that one global rank is not ideal. Unlike these works, we do not propose a new allocation scheme. We ask how much a single global rank matters across task types once other factors are held fixed.

Intrinsic dimension of fine-tuning. Aghajanyan et al. [2021] show that fine-tuning can be done in a random subspace of low dimension, and that this dimension shrinks as models grow. Effective-rank measures such as the entropy-based effective rank [Roy and Vetterli, 2007] quantify how spread out a spectrum is. Building on these ideas, we test whether the spectrum of the full-weight gradient at initialization predicts how much LoRA rank a task needs, and whether the needed rank shrinks from 0.5B to 1.5B parameters.

Rank, scaling, and learning rate. Kalajdzievski [2023] show that the standard α/r scaling makes high-rank adapters learn too slowly, and they propose $\alpha/\sqrt{r}$ scaling (rsLoRA). Under the common $\alpha = 2r$ convention we study, the scale is constant and the update magnitude instead grows with rank. Both results imply that rank and effective learning rate are coupled. We measure this coupling directly with a joint rank $\times$ learning-rate sweep and show that it explains most of the apparent task dependence of rank.

When does high rank help? Biderman et al. [2024] find that LoRA underperforms full fine-tuning on code and math, and that higher ranks narrow the gap, mainly in continued pre-training on billions

Table 1: Tasks. Each training set has 4,000 examples. Classification accuracy takes the argmax over the first tokens of the label words. Generative tasks use greedy decoding.

Task	Training source	Evaluation set	Metric
SST-2 [Socher et al., 2013, Wang et al., 2018]	GLUE SST-2 train	validation, 872	accuracy
EMOTION (6-way) [Saravia et al., 2018]	dair-ai/emotion train	test, 1,000 sampled	accuracy
AG NEWS (4-way) [Zhang et al., 2015]	train	test, 1,000 sampled	accuracy
BOOLQ [Clark et al., 2019]	train	validation, 1,000 sampled	accuracy
GSM8K [Cobbe et al., 2021]	train (calculator annotations removed)	test, 500 sampled	exact match of final number ($\leq$320 new tokens)
CODE	python_code_instructions_18k_alpaca	MBPP test, all 500 [Austin et al., 2021]	pass@1 (all 3 asserts pass, 10 s timeout)

of tokens. This finding is a main source of the intuition that code and math need high rank. We test the intuition in a different and very common regime: supervised fine-tuning (SFT) of small base models on a few thousand examples. In this regime we find the opposite ordering.

3 Methodology

3.1 Problem formulation

For a task t, a model m, and a LoRA rank $r \in \mathcal{R} = \{1, 2, 4, 8, 16, 32, 64, 128\}$, let $s_{t,m}(r)$ be the seed-averaged evaluation metric after fine-tuning. We pre-register three quantities. The **best rank** is $r^\star = \arg\max_{r \in \mathcal{R}} s_{t,m}(r)$. The **knee rank** is the smallest rank whose metric is within δ of the best,

$$r_{\text{knee}} = \min\{\, r \in \mathcal{R} : s_{t,m}(r) \geq s_{t,m}(r^\star) - \delta \,\}, \qquad \delta = \max(0.5\,\text{pp},\ \text{SE}(r^\star)), \tag{1}$$

where $\text{SE}(r^\star)$ is the standard error of the item-level mean at the best rank. The **rank-8 gap** is $s_{t,m}(r^\star) - s_{t,m}(8)$. We also report an NLL knee: the smallest rank that achieves 95% of the maximal reduction in held-out target NLL relative to zero-shot.

3.2 Tasks and models

We use the base (not instruction-tuned) QWEN2.5-0.5B and QWEN2.5-1.5B models [Qwen Team, 2024]. Both share a tokenizer and training recipe, so they give a clean size comparison. A 3B model was not feasible on our shared GPUs. Table 1 lists the six tasks. All tasks are cast as causal-LM SFT, with the loss applied only to response tokens. Each task uses 4,000 training examples for one epoch. Every run also reports held-out NLL on target tokens for 300 unseen examples from the training distribution. This is a lower-noise secondary metric.

3.3 Training recipe

We attach LoRA adapters to all linear layers (query, key, value, output, gate, up, and down projections) with $\alpha = 2r$. This is the "r=8, α=16" convention, and it gives a constant scale of 2. We use dropout 0.05, AdamW with learning rate 2×10^{-4}, a cosine schedule with 3% warmup, effective batch size 16, one epoch (250 steps), maximum length 512, bf16 autocast, gradient checkpointing, and gradient clipping at 1.0. The trainable parameter count ranges from 0.55M (r=1) to 70M (r=128) on 0.5B, and from 1.15M to 148M on 1.5B. We also evaluate the zero-shot base model (r=0). On 0.5B we use seeds 0 and 1. The seed controls the LoRA initialization, data order, and dropout; the training subset is fixed. On 1.5B we use seed 0 only.

Learning-rate sweep. Comparing ranks at a single learning rate is confounded, because under $\alpha = 2r$ the update $\Delta \boldsymbol{W} = 2\boldsymbol{B}\boldsymbol{A}$ is a sum of r rank-1 terms that are all trained at the same learning rate. A larger rank therefore acts roughly like a larger step. To remove this confound, we sweep the learning rate over $\{5 \times 10^{-5}, 10^{-4}, 2 \times 10^{-4}, 5 \times 10^{-4}\}$ for ranks $\{2, 8, 32, 128\}$ on CODE, GSM8K, and EMOTION (0.5B, seed 0). We extend the grid to 2×10^{-5} for GSM8K and CODE at $r \in \{8, 128\}$, and to 10^{-3} and 2×10^{-3} for EMOTION at $r \in \{2, 8\}$.

3.4 Gradient intrinsic dimensionality predictor

For each task and model, we accumulate the full-weight gradient $\boldsymbol{G}_\ell = \nabla_{\boldsymbol{W}_\ell} \mathcal{L}$ of the mean token loss at the pretrained weights over 128 training examples. We do this for all target matrices (168 for 0.5B, 196 for 1.5B) and compute each matrix's singular values $\sigma_1 \geq \sigma_2 \geq \ldots$. From each spectrum we compute r_p, the number of directions needed to capture a fraction p of $\|\boldsymbol{G}_\ell\|_F^2$, for

Table 2: Evaluation metric (%) vs. LoRA rank at learning rate 2×10^{-4}. 0.5B values are the mean of 2 seeds (seed SD $\leq$1.3 pp except BOOLQ r=128, 5.7 pp); 1.5B values use 1 seed. r=0 is the zero-shot base model. The best adapter rank per row is in **bold**. Classification curves are flat after r=8, EMOTION keeps rising, and GSM8K and code peak at low rank.

		Zero-shot	**LoRA rank r**							
Model	**Task**	r=0	1	2	4	8	16	32	64	128
0.5B	SST-2	88.5	91.7	91.7	92.4	92.8	92.9	**93.3**	92.5	91.5
	EMOTION	52.6	77.7	81.6	86.3	89.7	91.2	91.7	92.1	**92.3**
	AG NEWS	75.9	89.3	89.7	90.1	90.7	90.7	91.0	**91.1**	90.4
	BOOLQ	62.9	74.2	74.4	74.9	76.4	77.3	**78.0**	76.5	67.1
	GSM8K	3.4	**37.3**	34.8	35.4	34.1	33.4	33.0	32.3	30.1
	CODE	27.0	27.8	28.4	27.9	27.7	**28.8**	27.6	24.5	22.7
1.5B	SST-2	54.2	95.0	95.1	95.0	95.3	**95.6**	95.4	94.7	94.5
	EMOTION	52.6	79.1	82.2	87.1	91.0	92.6	**93.2**	92.9	92.3
	AG NEWS	77.8	90.1	89.8	90.1	90.8	90.7	91.4	90.8	**91.5**
	BOOLQ	67.6	81.6	82.9	83.7	84.4	84.4	84.3	**85.5**	83.0
	GSM8K	9.6	58.8	58.2	60.0	60.0	59.6	**60.6**	58.2	55.0
	CODE	47.2	**46.2**	45.2	44.8	44.0	45.0	44.0	42.8	40.6

$p \in \{0.5, 0.9, 0.99\}$. We also compute the entropy effective rank (ERANK) [Roy and Vetterli, 2007], the stable rank $\|\boldsymbol{G}_\ell\|_F^2/\sigma_1^2$, and the fraction of energy in the top-8 directions. The task-level predictor is the median over matrices, with r_{90} pre-registered as primary. The predictor is cheap: it needs one backward pass per example and no training.

3.5 Baselines and statistics

Our baselines are the zero-shot model, the r=8 default, and the best-rank oracle. For the predictor, the baselines are a constant prediction of r=8 and the zero-shot error rate as a naive difficulty score. We test the rank-8 gap with a paired bootstrap over evaluation items (5,000 resamples) and apply Holm correction over the six tasks. Choosing the best rank and measuring its gap on the same data is biased upward (winner's curse). We therefore also report a **cross-fitted gap**: we pick $r^\star$ on one seed and measure the gap on the other. For H4 we report Spearman ρ over the 12 (task, model) pairs with a permutation p-value (5,000 permutations). We also report a leave-one-out factor-2 hit rate: we fit a single multiplicative scale on 11 pairs and check whether the prediction for the held-out pair is within 2$\times$ of its knee. For H3 we use a sign test.

Compute. All runs use 4$\times$ RTX A6000 GPUs shared with other users. Our jobs ran in 2.5–5 GB slices through a memory-aware queue. A run took between 1.5 minutes (classification) and about 45 minutes (1.5B generation on a busy GPU), for about 4 hours of wall-clock time in total. We use PyTorch, Hugging Face `transformers` and `peft`, and SciPy.

4 Results

We first report rank curves at a fixed learning rate (section 4.1) and the cost of r=8 (section 4.2). We then show that the learning rate explains most of the rank effect (section 4.3), and we close with model size (section 4.4) and the gradient predictor (section 4.5).

4.1 Rank curves at a fixed learning rate

Table 2 and figure 1 show the main grid. **Does the best rank vary across tasks?** Yes: it ranges from r=1 (0.5B GSM8K, 1.5B code) to r=128 (0.5B EMOTION, 1.5B AG NEWS). But the task ordering is not the one the folk wisdom predicts.

Classification is flat. On SST-2 and AG NEWS, r=1 is already within 1.3 pp of the best rank for 0.5B, and the curve barely moves after r=8.

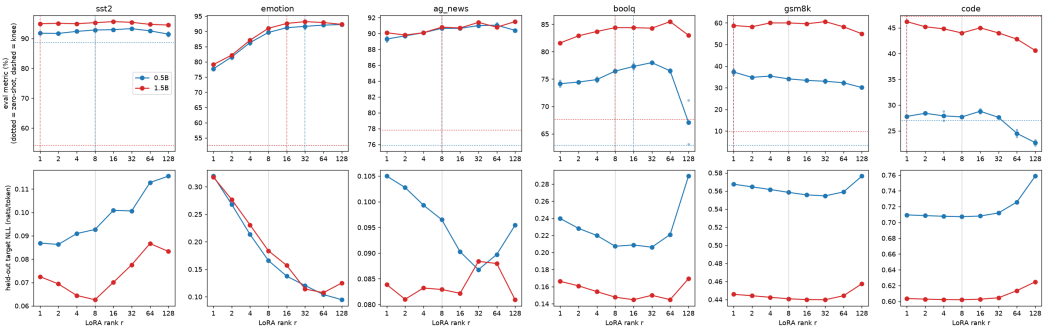


Figure 1: Rank curves at learning rate 2×10^{-4}. *(Top)* Evaluation metric vs. LoRA rank (lines are seed means, faint dots are individual seeds, dotted lines are zero-shot, dashed vertical lines are knees, grey line marks r=8). *(Bottom)* Held-out target NLL. Only EMOTION has a steep, monotone rank curve. GSM8K and code decline with rank, and their NLL worsens at $r \geq 64$. Curve shapes are nearly identical for 0.5B (blue) and 1.5B (red).

Table 3: Knee ranks and the cost of the r=8 default at learning rate 2×10^{-4}. Gap = best $-$ r=8 (pp), with a paired-bootstrap 95% CI over evaluation items. "Best r>8" is the gain of the best rank above 8 over r=8. Cross-fitted gaps choose the best rank on one seed and score it on the other (0.5B only). p_{Holm} is Holm-adjusted over tasks. Rows with $p_{\text{Holm}} < 0.05$ are in **bold**. Only EMOTION gains significantly from a rank above 8. The GSM8K and 1.5B code gaps favour r=1, i.e., a rank *smaller* than 8.

Model	Task	Best r	Knee	NLL knee	Gap (pp)	95% CI	Cross-fit	Best r>8 $-$ r8	p_{Holm}
0.5B	SST-2	32	8	1	0.5	[−0.5, 1.4]	0.5	0.5	1.0
	EMOTION	**128**	**32**	**1**	**2.6**	**[1.2, 4.0]**	**1.8**	**2.6**	**<0.001**
	AG NEWS	64	8	1	0.4	[−0.4, 1.1]	0.3	0.4	1.0
	BOOLQ	32	16	2	1.6	[−0.1, 3.1]	0.9	1.6	0.454
	GSM8K	1	1	1	3.2	[0.7, 5.7]	3.2	−0.7	0.132
	CODE	16	1	2	1.1	[−0.3, 2.5]	0.7	1.1	0.89
1.5B	SST-2	16	1	1	0.3	[−0.5, 1.3]	–	0.3	1.0
	EMOTION	**32**	**16**	**1**	**2.2**	**[0.8, 3.6]**	–	**2.2**	**0.018**
	AG NEWS	128	8	1	0.7	[−0.5, 1.9]	–	0.7	1.0
	BOOLQ	64	8	1	1.1	[−0.6, 2.8]	–	1.1	1.0
	GSM8K	32	1	1	0.6	[−2.6, 3.8]	–	0.6	1.0
	CODE	1	1	1	2.2	[0.2, 4.2]	–	1.0	0.324

Emotion is the one steep curve. 6-way EMOTION rises from 77.7% at r=1 to 89.7% at r=8 and 92.3% at r=128 on 0.5B. 1.5B follows the same shape.

Math and code prefer low rank. GSM8K accuracy falls almost monotonically with rank on 0.5B: 37.3% at r=1, 34.1% at r=8, and 30.1% at r=128. Held-out NLL stays nearly flat (0.555–0.568) up to r=64 and then worsens. On CODE, fine-tuning barely beats zero-shot on 0.5B (best 28.8% at r=16 vs. 27.0%), and r=128 drops to 22.7%, below zero-shot. On 1.5B, every code adapter scores below the 47.2% zero-shot pass@1.

Large ranks can collapse. BOOLQ rises to 78.0% at r=32 and then drops to 67.1% at r=128 (seeds: 63.1% and 71.1%). Together with GSM8K and code, large ranks at the default learning rate cost 7–11 pp on three of six tasks for 0.5B.

4.2 How much does r=8 cost?

Table 3 tests the rank-8 gap. **H1 (rank needs vary $\geq 8\times$)** holds numerically: knees range from 1 to 32. However, the direction is reversed. GSM8K and code have the *lowest* knees (r=1 for both models), and EMOTION has the highest (r=32 for 0.5B, r=16 for 1.5B).

H2 (r=8 costs 3–8 pp on high-rank tasks) is not supported. The largest gain from any rank above 8 is +2.6 pp on 0.5B EMOTION (CI [1.2, 4.0], $p_{\text{Holm}} < 0.001$). It shrinks to +1.8 pp when cross-fitted across seeds, which removes the winner's-curse bias. On 1.5B EMOTION the gain is +2.2 pp

Table 4: Rank × learning-rate sweep on QWEN2.5-0.5B (seed 0). Metric in %. The best value in each (task, rank) column is in **bold**. "Best over lr" takes the per-rank maximum. With a tuned learning rate, the spread across ranks is 0.7 pp on EMOTION, 1.8 pp on CODE, and 3.6 pp on GSM8K (favouring r=8 over r=128).

	EMOTION				GSM8K				CODE			
lr	$r2$	$r8$	$r32$	$r128$	$r2$	$r8$	$r32$	$r128$	$r2$	$r8$	$r32$	$r128$
2×10^{-5}	–	–	–	–	–	36.2	–	**34.0**	–	28.4	–	**27.8**
5×10^{-5}	68.9	77.5	87.0	92.1	35.8	**37.6**	**35.0**	30.2	28.6	28.4	**29.6**	**27.8**
1×10^{-4}	74.7	84.8	91.0	**92.4**	**36.0**	35.6	33.0	33.0	**29.0**	**28.8**	28.6	25.2
2×10^{-4}	82.2	89.6	**92.4**	**92.4**	35.2	34.0	32.2	30.0	28.4	27.8	27.6	22.2
5×10^{-4}	90.3	**92.8**	**92.4**	90.5	35.0	32.2	31.2	22.4	**29.0**	26.6	22.8	15.4
1×10^{-3}	**92.1**	**92.8**	–	–	–	–	–	–	–	–	–	–
2×10^{-3}	91.7	91.0	–	–	–	–	–	–	–	–	–	–
Best over lr	92.1	92.8	92.4	92.4	36.0	37.6	35.0	34.0	29.0	28.8	29.6	27.8

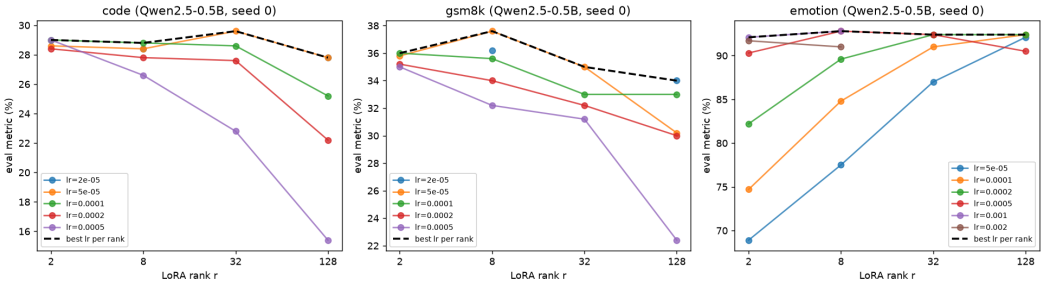


Figure 2: Metric vs. rank for each learning rate on QWEN2.5-0.5B (seed 0). The dashed black line takes the best learning rate per rank. *(Right)* On EMOTION, low learning rates produce a steep "needs high rank" curve, while learning rates of 5×10^{-4}–10^{-3} give a flat curve. *(Left)* and *(Center)* On CODE and GSM8K, high rank never helps, and high rank combined with a high learning rate is destructive.

($p_{\text{Holm}} = 0.018$). These are the only significant gaps after correction. No generative task gains more than 1 pp from a rank above 8. On 0.5B GSM8K the best rank above 8 is 0.7 pp *worse* than r=8. The two largest uncorrected gaps, 0.5B GSM8K (+3.2 pp, CI [0.7, 5.7]) and 1.5B code (+2.2 pp, CI [0.2, 4.2]), both favour r=1. On these tasks, r=8 is too large, not too small.

4.3 The rank effect is mostly a learning-rate effect

Table 4 and figure 2 show the rank × learning-rate sweep. **Does the EMOTION gap survive learning-rate tuning?** No. At learning rate 5×10^{-5}, EMOTION accuracy climbs from 68.9% (r=2) to 92.1% (r=128), a textbook "needs high rank" curve. At 5×10^{-4}–10^{-3} the curve is flat: 92.1% (r=2), 92.8% (r=8), 92.4% (r=32), 92.4% (r=128). The best-over-lr spread across ranks is only 0.7 pp, and r=8 is the best rank. By contrast, the learning rate changes r=8 accuracy by 15.3 pp (77.5% to 92.8%). The explanation is the $\alpha = 2r$ scaling: the update $2\boldsymbol{BA}$ is a sum of r rank-1 terms trained at the same learning rate, so a larger rank acts like a larger step. A low rank at a higher learning rate reaches the same accuracy.

On GSM8K, the best-over-lr accuracy is highest at r=8 (37.6%, learning rate 5×10^{-5}) and lowest at r=128 (34.0%, 2×10^{-5}). High rank never helps, and at a high learning rate it is destructive (r=128 at 5×10^{-4}: 22.4%). On CODE, best-over-lr pass@1 is flat (27.8–29.6%) and within noise (SE ≈ 2 pp). r=128 at 5×10^{-4} falls to 15.4%, with held-out NLL rising from 0.708 to 0.857.

The best learning rate shrinks as rank grows on all three tasks: on EMOTION from 10^{-3} (r=2) to 10^{-4}–2×10^{-4} (r=128), on GSM8K from 10^{-4} to 2×10^{-5}, and on CODE from 10^{-4}/5×10^{-4} to 2×10^{-5}/5×10^{-5}. This is the coupling expected under $\alpha \propto r$. The common recipe of "r=8

Table 5: Effect of model size at learning rate 2×10^{-4}. The knee is lower for 1.5B on 3 tasks, equal on 3, and never higher (sign test $p = 0.25$). The clearest size effect is robustness to large ranks: on BOOLQ, r=128 costs 10.9 pp at 0.5B but 2.5 pp at 1.5B.

	Knee		**Best rank**		$r128-$ **best (pp)**	
Task	0.5B	1.5B	0.5B	1.5B	0.5B	1.5B
SST-2	8	**1**	32	16	−1.8	−1.1
EMOTION	32	**16**	128	32	0.0	−0.9
AG NEWS	8	8	64	128	−0.7	0.0
BOOLQ	16	**8**	32	64	−10.9	−2.5
GSM8K	1	1	1	32	−7.2	−5.6
CODE	1	1	16	1	−6.1	−5.6

Table 6: Gradient spectrum statistics at initialization (median over target matrices). r_{90}, r_{99}: directions needed for 90% / 99% of gradient energy. ERANK: entropy effective rank. Top-8: fraction of energy in the top 8 directions. The ordering code $\gg$ GSM8K $>$ BOOLQ $>$ AG NEWS $>$ EMOTION $>$ SST-2 is consistent across sizes.

	QWEN2.5-0.5B				**QWEN2.5-1.5B**		
Task	r_{90}	r_{99}	ERANK	Top-8	r_{90}	r_{99}	ERANK
SST-2	2	4	7.4	0.999	2	5	10.2
EMOTION	2	5	10.3	0.999	2	5	12.3
AG NEWS	2	5	17.0	0.997	2	5	18.6
BOOLQ	2	7	26.1	0.993	2	6.5	22.4
GSM8K	4	9	46.3	0.990	4	11	80.4
CODE	6	143	223.7	0.919	3	89	244.2

with learning rate 2×10^{-4}" is a matched pair. Changing the rank without changing the learning rate creates apparent rank effects.

4.4 Does the knee shrink with model size?

Table 5 compares the two model sizes. **H3** is directionally consistent but not statistically supported. The knee is lower for 1.5B on SST-2, EMOTION, and BOOLQ, equal on the other three tasks, and never higher. A sign test on the three non-tied pairs gives $p = 0.25$. The best rank moves in both directions. The fixed-learning-rate curves have nearly the same shape at both sizes (figure 1). The clearest size effect is robustness: 1.5B suffers much less from the r=128 collapse on BOOLQ (-2.5 pp vs. -10.9 pp).

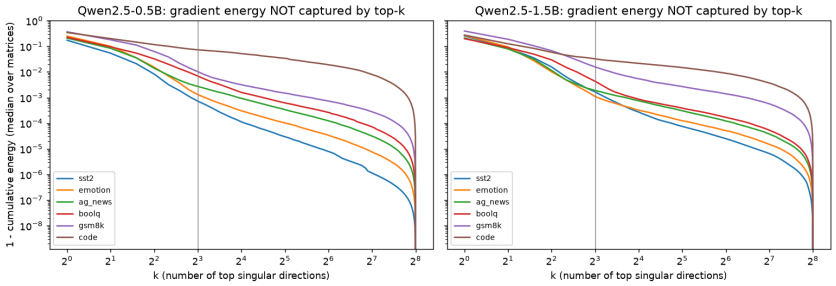
4.5 Does the initial gradient spectrum predict rank needs?

Table 6 and figure 3a show that the gradient statistic separates tasks cleanly and consistently across model sizes. The code gradient spreads over two orders of magnitude more directions than SST-2 (ERANK 223.7 vs. 7.4 on 0.5B), and only 91.9% of its energy lies in the top 8 directions, compared with 99.9% for SST-2.

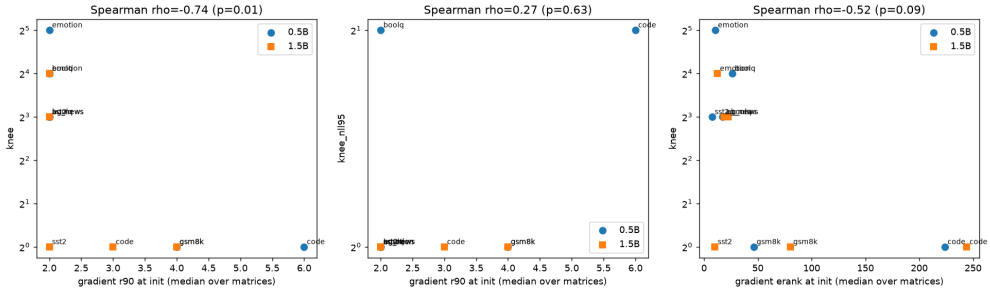
H4 (the statistic predicts the knee with $\rho > 0.7$) is refuted, with a significant correlation of the opposite sign (table 7, figure 3b). The primary predictor r_{90} has $\rho = -0.74$ ($p = 0.014$) with the knee. All other gradient statistics also correlate negatively (ρ from -0.52 to -0.63). With a leave-one-out calibrated scale, r_{90} lands within $2\times$ of the knee for 0 of 12 pairs, while always predicting r=8 succeeds for 6 of 12. The NLL knee shows a weak positive correlation ($\rho = 0.27$, $p = 0.63$). Its high hit rate (0.92) is not informative, because NLL knees are 1 or 2 for every pair.

5 Discussion

Table 8 summarizes our verdicts. We now discuss why the results run against the common intuition.



(a) Gradient energy outside the top-k directions



(b) Gradient statistic vs. empirical knee

Figure 3: The gradient-spectrum predictor. *(a)* Fraction of gradient energy not captured by the top-k singular directions, median over target matrices, for 0.5B (left) and 1.5B (right). Spectra are truncated at 256, so curves drop to 0 there by construction. The grey line marks k=8. Code has by far the heaviest tail. *(b)* r_{90} (left), r_{90} vs. NLL knee (middle), and ERANK (right) against the knee rank (log scale). Tasks with the heaviest gradient tails (code, GSM8K) have the *lowest* knees.

Why do math and code not need high rank here? We see three explanations that fit the data. First, **SFT on 4,000 examples mostly teaches format, not capability**. Qwen2.5 base models saw a lot of math and code in pre-training [Qwen Team, 2024]. Zero-shot GSM8K accuracy is 3.4% mainly because the base model does not emit the expected final-answer format, and r=1 already reaches 37.3%. MBPP gains are at most 2 pp. Learning a response format is a low-rank change. The high-rank regime of Biderman et al. [2024] involves billions of tokens of continued pre-training, which is a different setting. Second, **a higher rank means a larger effective step and more forgetting**. Under $\alpha = 2r$, large ranks at a fixed learning rate overfit the small SFT set and damage pretrained skills. Held-out NLL worsens at $r \geq 64$ on GSM8K and code, and every collapse we observe coincides with worse NLL. Third, **EMOTION is the exception because its labels are idiosyncratic**. Distinctions such as "love" vs. "joy" are noisy and dataset-specific, and the base model does not already encode them (52.6% zero-shot). EMOTION therefore needs the most movement away from the base model, and at a fixed learning rate only more rank delivers that movement.

Why does the gradient predictor fail? At initialization, the mean gradient is dominated by one or two directions in every task: the median stable rank is 1.2–1.7, and $r_{50} = 1$. The tail that r_{90} and ERANK measure reflects how diverse the token-level targets are. Code has the most varied output tokens, and SST-2 has a single label token. Diverse targets do not mean that the *solution* needs many directions: the best code and GSM8K adapters have rank 1–16. The predictor also ignores the rank–learning-rate coupling, which dominates the empirical knees. A useful predictor probably needs information from training, such as the spectrum of $\Delta \boldsymbol{W}$ after a short pilot run.

Failure modes. The high-rank collapses (BOOLQ r=128, code r=128, GSM8K r=128 at learning rate 5×10^{-4}) all come with worse held-out NLL. They reflect overfitting or instability, not a lack of capacity. On 1.5B code, every adapter is below the zero-shot MBPP score, because the alpaca-style training instructions do not match MBPP's prompt and assert format. The 1.5B code knee therefore mostly measures how little damage each rank does.

Table 7: How well initial-gradient statistics predict rank needs over $n = 12$ (task, model) pairs. Positive ρ means "predicts higher rank" (top-8 energy and zero-shot accuracy are sign-flipped). "Within $2\times$" is the leave-one-out hit rate after fitting one multiplicative scale. All significant correlations with the knee are *negative*, and no gradient statistic beats always predicting $r{=}8$ (best value per target in **bold**).

Target	Predictor	Spearman ρ	Perm. p	Within $2\times$
Knee	r_{90} (primary)	-0.74	0.014	0.00
	r_{99}	-0.58	0.059	0.00
	ERANK	-0.52	0.093	0.08
	Top-8 energy	-0.63	0.029	–
	Stable rank	-0.57	0.061	0.00
	Zero-shot accuracy	-0.50	0.111	–
	Constant $r{=}8$	–	–	**0.50**
Best rank	r_{90}	-0.64	0.029	**0.42**
	ERANK	-0.45	0.143	0.08
	Constant $r{=}8$	–	–	0.17
NLL knee	r_{90}	0.27	0.632	**0.92**
	r_{99}	0.47	0.161	0.67

Table 8: Verdicts on the pre-registered hypotheses.

ID	Verdict	Evidence
H1	Refuted in substance	Knees span 1–32 at a fixed learning rate, but code and GSM8K are lowest and EMOTION is highest. Best-over-lr curves are flat or favour low rank.
H2	Not supported	The maximum gain above $r{=}8$ is 2.6 pp (cross-fitted 1.8 pp), only on EMOTION, and it vanishes with a tuned learning rate.
H3	Weak / inconclusive	The knee is lower for 1.5B on 3/6 tasks and never higher (sign test $p = 0.25$). Best ranks move both ways.
H4	Refuted (opposite sign)	$\rho(r_{90}, \text{knee}) = -0.74$, $p = 0.014$. Factor-2 hit rate 0/12 vs. 6/12 for constant $r{=}8$.

Limitations. Our models are small (0.5B and 1.5B) and come from one family. The 1.5B grid has one seed, so H3 rests on two sizes and is underpowered. The results may not transfer to 7B+ models or other families. We use 4,000 examples and one epoch per task. Rank needs may grow with much larger SFT sets or with continued pre-training. We study one parameterization ($\alpha = 2r$ on all linear layers). rsLoRA scaling [Kalajdzievski, 2023] or attention-only targets would change the rank–learning-rate coupling. The learning-rate sweep covers only 3 tasks, 4 ranks, one model, and one seed. Evaluation sets of 500–1,000 items give standard errors of 1–2 pp, which is comparable to many differences between neighbouring ranks. We therefore base our conclusions on monotone trends and learning-rate interactions rather than on single-rank comparisons. The gradient predictor used 128 examples and one seed, and we did not test alternatives such as per-example gradient covariance. Finally, we used Hugging Face generation with greedy decoding, and timings are not comparable across runs because the GPUs were shared.

Broader implications. For practitioners, our results suggest a simple rule: sweeping the learning rate is worth much more than sweeping the rank. Keeping $r \approx 8$–16 also saves memory and compute compared with large ranks. A negative result like ours can also prevent wasted effort on rank searches. We see no direct negative societal impact from this study beyond the general risks of fine-tuning language models.

6 Conclusion

We asked whether the default LoRA rank of 8 is a blind spot for tasks such as math and code. In a controlled sweep of about 200 adapters on QWEN2.5-0.5B and QWEN2.5-1.5B, the best rank was not a stable property of the task. At a fixed learning rate it ranged from 1 to 128, but the ordering contradicted the intuition: GSM8K and code preferred low rank, and only 6-way EMOTION classifi-

cation gained significantly from more rank (+2.6 pp). Once we tuned the learning rate per rank, that gain disappeared, and r=8 was the best rank or within 0.8 pp of it on all three tasks in our learning-rate sweep. The spectrum of the initial gradient ordered tasks consistently but anti-predicted the needed rank ($\rho = -0.74$, $p = 0.014$).

The key takeaway is that, in small-data SFT, the default rank is not the blind spot; the default learning rate at a non-default rank is. We recommend keeping $r \approx 8$–16, lowering the learning rate as rank grows (roughly $\propto 1/\sqrt{r}$ to $1/r$ under $\alpha = 2r$), and avoiding $r \geq 64$ with standard scaling on small SFT sets.

Future work should (1) repeat the study at 7B+ scale and with $\geq$100k-example or continued pre-training data, where high-rank benefits have been reported; (2) run the full rank $\times$ learning-rate grid with rsLoRA scaling; (3) test predictors based on a short pilot run, such as the spectrum of $\Delta \boldsymbol{W}$ after 20 steps; and (4) use more seeds and larger evaluation sets for the generative tasks.

References

Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL-HLT*, pages 2924–2936, 2019.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *15th European Signal Processing Conference*, pages 606–610, 2007.

Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of EMNLP*, pages 3687–3697, 2018.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.